

ES6 · JavaScript

Object-Oriented Programming

A Complete Introduction to the Four Pillars of OOP:
Problem & Solution

ECMAScript Course | Prepared by: Mohamed | ITI

Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm built around the idea of bundling data and the behavior that operates on that data into a single unit called a **Class**, from which we create instances called **Objects**. The core goal is to model real-world problems in code in a way that's closer to how things actually work, instead of writing everything as scattered functions and variables with no real logical connection between them.

Before OOP, Procedural Programming faced a fundamental problem: data and the functions that operate on it were completely separate. This created several issues — difficulty maintaining the code, duplicated logic, and difficulty tracking who was modifying the data. OOP was introduced to solve exactly this, and it's built on four core principles.



The Four Pillars of OOP

Encapsulation

Bundling data (fields) and the methods that operate on them inside a single class, while controlling who is allowed to access that data from the outside.

The Problem

If all fields are public, any part of the program can modify them directly and without control, which threatens data integrity. There's no protection at all.

The Solution

Make fields private using #, and provide controlled access through accessors (get / set) or methods.

```
class BankAccount {
  #balance; // private field

  constructor(owner, balance) {
    this.owner = owner; // public field
    this.#balance = balance;
  }

  get balance() {
    return this.#balance; // controlled read access
  }

  deposit(amount) {
    if (amount > 0) this.#balance += amount;
  }
}

const acc = new BankAccount("Ahmed", 1000);
acc.deposit(500);
console.log(acc.balance); // 1500
// acc.#balance Error: private, cannot be accessed from
// outside
```

Abstraction

Hiding complex implementation details from the user, and exposing a simple, clear interface for interacting with the object.

The Problem

If the user of a class has to know every internal detail and complex step just to use it, the code becomes hard to use and error-prone.

The Solution

Expose one simple method that summarizes all the complex details internally, so the user just calls it without needing to know exactly how it works.

```
class CoffeeMachine {
  #grindBeans() { /* complex details */ }
  #heatWater() { /* complex details */ }
  #brew() { /* complex details */ }

  .
  makeCoffee() { // the simple public interface
    this.#grindBeans();
    this.#heatWater();
    this.#brew();
    return "  Your coffee is ready";
  }
}

const machine = new CoffeeMachine();
console.log(machine.makeCoffee());
// The user doesn't need to know how grinding or heating
works
```

Inheritance

Allowing a new class (child) to acquire the properties and behaviors of an existing class (parent), then extend or override them.

The Problem

If several classes share the same properties and methods (e.g. Person, Student, Employee), and each one rewrites that same code from scratch, that's heavy duplication and a maintenance nightmare.

The Solution

Use `extends` so the child class inherits from the parent, and `super()` to call the parent's constructor.

```
class Person {
  constructor(id, name) {
    this.ID = id;
    this.Name = name;
  }
}

class Student extends Person {
  constructor(id, name, track) {
    super(id, name); // must be called first
    this.track = track;
  }
}

const s = new Student(100, "Ahmed", "Web Development");
console.log(s.Name, s.track); // Ahmed Web Development
```

Polymorphism

The ability for the same method to behave differently depending on the type of object calling it, while keeping the same name and interface.

The Problem

If each type of object needs different behavior for the same operation, you end up writing a lot of if/else conditions to distinguish between types, which is hard to extend and maintain.

The Solution

Each subclass overrides the same method with its own implementation, and we just call it without worrying about the exact type.

```

class Employee {
  constructor(name) { this.name = name; }
  work() { return `${this.name} is working`; }
}

class DotnetDeveloper extends Employee {
  work() { return `${this.name} is writing dotnet code`; }
}

class PythonDeveloper extends Employee {
  work() { return `${this.name} is writing python code`; }
}

const team = [
  new DotnetDeveloper("Sara"),
  new PythonDeveloper("Omar"),
];

team.forEach(emp => console.log(emp.work()));
// Sara is writing dotnet code
// Omar is writing python code
// Same .work() call, different behavior per type

```

Quick Summary

Principle	Solves the Problem of	ES6 Tool
Encapsulation	Unprotected, freely modifiable data	#field + get/set
Abstraction	Overly complex usage	Simple public methods hiding private methods
Inheritance	Duplicated code across similar classes	extends + super()
Polymorphism	Excessive if/else logic based on type	Method Overriding

Prepared as an introductory summary for the ES6 Classes lesson, part of the
ECMAScript course